

Checkbox Mini-Projects

Project Drisya Decision Logic

Suwarna Pyakurel

Checkbox Mini-Projects For Decision Logic

Each mini-project should become one GitHub milestone or one clean LinkedIn progress post. Build them in order.

Mini-Project 1: Blackboard Contract And State Snapshot

Goal: Convert raw Blackboard data into a safe, typed decision state.

- ☐ Define Blackboard key constants in `contracts.py`.
- ☐ Define payload types for pose, detection, proximity, mission state, path, and frontier.
- ☐ Implement `DecisionStateSnapshot`.
- ☐ Validate missing fields.
- ☐ Validate stale timestamps.
- ☐ Validate detection confidence range from 0.0 to 1.0.
- ☐ Validate pose fields: `x`, `y`, `theta_deg`.
- ☐ Add tests for valid and invalid states.

Acceptance criteria:

- ☐ Invalid Blackboard data cannot silently produce movement.
- ☐ Stale sensor data is flagged.
- ☐ Tests run without simulator or hardware.

GitHub/LinkedIn line:

↳ Built a typed Blackboard validation layer for a modular autonomous SAR robot, preventing unsafe decisions from stale or malformed sensor data.

Mini-Project 2: Safety-First Behavior Tree

Goal: Build deterministic safety behavior before victim search or RL.

- ☐ Add `py_trees` to dependencies when implementation starts.
- ☐ Implement `IsEmergencyStopRequested`.
- ☐ Implement `IsObstacleTooClose`.
- ☐ Implement `IsDataStale`.
- ☐ Implement `WriteStopCommand`.
- ☐ Implement `WriteSafeCommand`.
- ☐ Build top-level priority selector.

☐ Test emergency stop override.

☐ Test obstacle safety override.

Acceptance criteria:

☐ Emergency stop always writes zero motor speeds.

☐ Obstacle safety overrides victim navigation.

☐ Stale data triggers safe stop.

GitHub/LinkedIn line:

ı Implemented a safety-first Behavior Tree where emergency stop and proximity checks override all adaptive robot behavior.

Mini-Project 3: WiFi Victim Detection Response

Goal: Turn WiFi detection results into mission intent.

☐ Read `detection/result`.

☐ Ignore stale detection data.

☐ Ignore low-confidence detections.

☐ Request confirmation for medium-confidence detections.

☐ Set victim target for high-confidence detections.

☐ Debounce noisy detections.

☐ Test low, medium, and high confidence cases.

Acceptance criteria:

☐ Confidence below 0.60 does not change target.

☐ Confidence from 0.60 to 0.75 requests confirmation.

☐ Confidence above 0.75 sets victim target.

GitHub/LinkedIn line:

ı Built confidence-gated victim response logic that converts WiFi CSI human detections into explainable navigation targets.

Mini-Project 4: Deterministic Exploration Baseline

Goal: Build a reliable exploration policy before training PPO.

☐ Consume navigation frontier data if available.

☐ Choose nearest valid high-score frontier.

☐ Avoid occupied cells.

☐ Add fallback behavior when no frontier exists.

☐ Write `decision/target` with `target_type="frontier"`.

☐ Test with fake 50 x 50 grids.

☐ Measure explored-cell coverage in simple scenarios.

Acceptance criteria:

☐ Occupied cells are never selected.

☐ Exploration only runs when mission is active.

☐ Baseline works without RL installed.

GitHub/LinkedIn line:

ı Developed a frontier-based exploration baseline for SAR robot decision-making with repeatable coverage and collision metrics.

Mini-Project 5: Runtime Brain Loop

Goal: Make the decision module runnable.

- ☐ Implement `brain.py`.
- ☐ Tick at 10 Hz.
- ☐ Read Blackboard state once per tick.
- ☐ Build `DecisionStateSnapshot`.
- ☐ Tick Behavior Tree.
- ☐ Write `state/motor_command`.
- ☐ Write `decision/state`.
- ☐ Write `decision/trace`.
- ☐ Add graceful shutdown.
- ☐ Add dry-run mock mode.

Acceptance criteria:

- ☐ Runs without simulator in dry-run mode.
- ☐ Handles invalid state safely.
- ☐ Exits cleanly on keyboard interrupt.

GitHub/LinkedIn line:

ı Built a 10 Hz robot decision loop integrating Blackboard reads, Behavior Tree ticks, motor command publishing, and explainability traces.

Mini-Project 6: Explainability Trace Logger

Goal: Make every robot decision auditable.

- ☐ Define trace schema.
- ☐ Log selected behavior.
- ☐ Log source layer: `BT_SAFETY`, `BT_MISSION`, `HEURISTIC_EXPLORATION`, `RL_POLICY`.
- ☐ Log important input facts.
- ☐ Log output command.
- ☐ Export JSONL or CSV traces.
- ☐ Write trace formatting tests.

Acceptance criteria:

- ☐ Every command has a reason.
- ☐ Trace entries are machine-readable.
- ☐ Dashboard can read latest behavior state.

GitHub/LinkedIn line:

ı Designed an explainability trace layer that maps every autonomous robot command to the behavior node and sensor facts that caused it.

Mini-Project 7: Gymnasium SAR Exploration Environment

Goal: Build an RL training environment after the baseline works.

- ☐ Add `gymnasium`.
- ☐ Implement `SARExploreEnv`.
- ☐ Define observation space.
- ☐ Define action space.
- ☐ Implement `reset`.
- ☐ Implement `step`.
- ☐ Reward explored cells.
- ☐ Reward confirmed target discovery.
- ☐ Penalize collision and unsafe proximity.
- ☐ Add random seed support.
- ☐ Run one full episode without crashing.

Acceptance criteria:

- ☐ Environment follows Gymnasium API.
- ☐ Reward components are inspectable.
- ☐ Environment does not replace runtime safety layer.

GitHub/LinkedIn line:

ı Created a Gymnasium-compatible SAR exploration environment for training and evaluating robot exploration policies.

Mini-Project 8: PPO Training And Baseline Comparison

Goal: Train PPO and compare it honestly against deterministic exploration.

- ☐ Add `stable-baselines3`.
- ☐ Implement `train_ppo.py`.
- ☐ Save checkpoints outside git or in ignored folder.
- ☐ Evaluate with fixed random seeds.
- ☐ Compare PPO against deterministic baseline.
- ☐ Report only measured simulation metrics.

Acceptance criteria:

- ☐ PPO has reproducible evaluation results.
- ☐ Metrics table includes baseline comparison.
- ☐ PPO never bypasses emergency safety.

GitHub/LinkedIn line:

ı Trained and evaluated a PPO exploration policy for a simulated SAR robot, benchmarked against a deterministic frontier baseline.

Mini-Project 9: Simulator Integration

Goal: Prove the module works with the team system.

- ☐ Run decision logic with simulator mock state.
- ☐ Confirm simulator reads motor command.
- ☐ Confirm dashboard reads decision state.
- ☐ Confirm dashboard can show decision trace.
- ☐ Test emergency stop scenario.
- ☐ Test obstacle scenario.
- ☐ Test victim detection scenario.

Acceptance criteria:

- ☐ Decision logic does not import simulator internals.
- ☐ Communication happens through Blackboard keys.
- ☐ Three demo scenarios are reproducible.

GitHub/LinkedIn line:

↳ Integrated a Behavior Tree decision module with a simulated SAR robot dashboard using a shared Blackboard architecture.

Mini-Project 10: Hardware-Ready Command Safety

Goal: Prepare output commands for Raspberry Pi/ESP32 integration.

- ☐ Freeze motor command schema.
- ☐ Clamp speed range.
- ☐ Limit command duration.
- ☐ Add watchdog stop behavior.
- ☐ Add stale sensor safe stop.
- ☐ Test JSON serialization.

Acceptance criteria:

- ☐ Invalid speed values cannot reach hardware.
- ☐ Commands cannot run forever.
- ☐ Missing sensor updates stop the robot.

GitHub/LinkedIn line:

↳ Prepared hardware-safe motor command outputs with clamping, duration limits, watchdog behavior, and stale-data fail-safes.